

WAAM Slicer

Wire Arc Additive Manufacturing — Path Planning, Process-Aware Slicing and Fanuc LS Generation

A detailed README for the current five-file implementation

This document explains how the slicer is actually organised and how the pieces fit together: STL loading, geometry analysis, layer generation, wall and infill planning, experimental bead-model integration, adaptive layer thickness, gap compensation, torch orientation, Fanuc LS generation, and post-generation inspection tools.

Project files covered

File	Role
waam_slicer.py	Core slicing engine and path-planning logic
waam_bead_models.py	DoE-derived bead height/width model
waam_slicer_ui.py	Tkinter desktop application
waam_ls_generator.py	Fanuc R-30iB LS program generator
waam_ls_tools.py	LS parser, visualiser and coordinate-offset editor

Important: this README reflects the current code supplied for this project. An older README exists in the file library, but several of its statements are now obsolete — notably its description of adaptive slicing and thin-wall support. The implementation described here takes precedence.

Generated from the project source on 12 August 2026.

Contents

- 1. What the slicer is trying to do
- 2. Project architecture
- 3. Installation and requirements
- 4. Quick start — the normal GUI workflow
- 5. The slicing pipeline, from STL to LayerData
- 6. Geometry analysis and parameter suggestions
- 7. Layer generation: constant and adaptive modes
- 8. The experimental bead model
- 9. Wall generation
- 10. Infill strategies
- 11. Gap / fill-mismatch compensation
- 12. Seam management and path ordering
- 13. Torch orientation and adaptive tilt
- 14. Fanuc LS generation
- 15. LS inspection and coordinate offsets
- 16. Outputs and reports
- 17. Configuration reference
- 18. Python API and programmatic use
- 19. Practical configuration recipes
- 20. Validation, limitations and safe deployment
- 21. Developer notes and extension points
- 22. Troubleshooting

1. What the slicer is trying to do

A conventional slicer mainly asks a geometric question: *where should the tool move so that the desired shape is reproduced?* WAAM makes that question more complicated because the tool does not deposit an infinitesimally thin line. The process deposits a relatively wide, tall bead, and the bead geometry changes with travel speed. The current slicer therefore treats geometry and process behaviour as one problem rather than two separate steps.

The central idea is deliberately simple: **slice the part, decide what has to be deposited, then use the available process model to make the deposition more realistic.** The core engine always works from the cross-section that actually exists at a layer. It computes wall paths first, then fills whatever remains. This is particularly important for tapered parts: the code does not rely on a global thin/thick classification to decide whether a layer should exist.

The resulting path is not the final robot program yet. The pipeline is:

```
STL
↓
mesh loading + Z alignment
↓
geometry analysis
↓
layer Z schedule
↓
horizontal mesh intersection
↓
world-XY polygon reconstruction
↓
wall paths
↓
remaining infill region
↓
raster / contour paths
↓
gap compensation + edge passes
↓
per-layer torch W/P/R + optional weld speed
↓
LayerData
↓
Fanuc LS generator
↓
.LS robot program
↓
LS parser / visualiser / offset editor
```

That last stage matters. The project does not stop at writing a text file; it includes tools for opening the generated LS program again, separating travel/weld/edge motions, viewing the build in 3D, checking the layer structure, and applying a coordinate offset when the robot-side setup requires one.

The implementation is therefore best thought of as a **WAAM path-planning and robot-code preparation tool**, not simply an STL slicer.

2. Project architecture

There are five Python files in the current implementation. They are intentionally separated so that geometry/path planning, process modelling, robot-code generation, and inspection can evolve independently.

Module	Main classes / objects	Responsibility
waam_slicer.py	SlicerConfig, LayerData, WAAMSlicer	STL loading, geometry analysis, slicing, walls, infill, mismatch compensation, seam handling, torch orientation, visualisation and reports.
waam_bead_models.py	BeadModel	Stores experimental speed–height/width data and provides forward and inverse interpolation.
waam_slicer_ui.py	WAAMSlicerUI	Desktop front end: parameters, file selection, geometry suggestions, slicing, visualisation, JSON export and LS generation.
waam_ls_generator.py	LSConfig, LSGenerator	Converts LayerData into Fanuc R-30iB LS motion blocks, weld commands, registers and position blocks.
waam_ls_tools.py	LSParser, LSVisualiser, LSOffsetEditor	Reads generated LS files, classifies motions, visualises them and applies X/Y/Z/W/P/R offsets.

The core data structure passed between the slicer and the LS generator is **LayerData**. Each layer stores its Z height, wall paths, infill paths, edge compensation passes, mismatch information, torch orientation, and — when adaptive bead-model slicing is active — the weld speed assigned to that layer.

```
LayerData
    ■■■ index
    ■■■ z
    ■■■ walls
    ■■■ infill_paths
    ■■■ edge_passes          # (LineString, speed)
    ■■■ mismatch_info
    ■■■ wpr                  # (W, P, R)
    ■■■ weld_speed_mms      # optional
```

This separation is useful: the slicer does not need to know how Fanuc syntax is written, and the LS generator does not need to know how a polygon was obtained from the STL.

3. Installation and requirements

The code imports NumPy, Trimesh, Shapely, Matplotlib and SciPy. The GUI uses Tkinter, which is normally included with a standard Python installation. The current source headers specify:

```
pip install trimesh numpy shapely matplotlib scipy
```

A practical environment is therefore:

Component	Recommendation
Python	A recent CPython 3.x installation
NumPy	Required
Trimesh	Required for STL loading, mesh sections and geometry operations
Shapely	Required for 2D polygon/path operations
Matplotlib	Required for slicer and LS visualisation
SciPy	Required by the current dependency set / numerical workflow
Tkinter	Required for the desktop GUI; normally bundled with Python

Keep all five project files together when using the GUI. The UI imports the core slicer, bead model, LS generator and LS tools by module name.

Recommended project folder:

```
WAAM_Slicer/  
■■■ waam_slicer.py  
■■■ waam_bead_models.py  
■■■ waam_slicer_ui.py  
■■■ waam_ls_generator.py  
■■■ waam_ls_tools.py
```

Run the desktop application with:

```
python waam_slicer_ui.py
```

The GUI checks whether `waam_bead_models.py` is importable and shows its status near the top of the window.

4. Quick start — the normal GUI workflow

The intended user experience is deliberately straightforward. You do not need to call the core classes manually for a normal slicing job.

Step 1 — Select the STL

Use **STL File Path** → **Browse**. The slicer loads the mesh and automatically translates it so that the minimum Z coordinate is zero. If the STL is a Trimesh Scene containing multiple meshes, those meshes are concatenated. A non-watertight mesh is not rejected automatically; instead, the slicer prints a warning because open geometry can produce incomplete sections.

Step 2 — Orient the part if necessary

The orientation viewer provides Rx, Ry and Rz sliders. The selected rotation is applied to a copy of the mesh, and the base can automatically be re-aligned to Z=0. Once confirmed, the rotated mesh is stored and the normal **Run Slicer** operation uses that mesh rather than reloading the original STL.

Step 3 — Analyse Geometry

This does not slice the part. It samples five Z levels across the middle 80% of the part height and reports a geometry type, approximate minimum/maximum wall thickness, annular fraction and multi-body occurrence. It also produces parameter suggestions.

Step 4 — Configure the process

The GUI groups settings into Layer Settings, Bead Geometry, Wall & Infill, Seam Control, Gap Compensation, Torch Orientation, Adaptive Tilt, LS Code Settings, Process Calculator and Output.

Step 5 — Run Slicer

The UI creates a `SlicerConfig`, loads either the confirmed oriented mesh or the STL, calls `slice()`, then calls `print_summary()`. The slice itself runs in a worker thread so the desktop interface remains responsive.

Step 6 — Inspect before generating robot code

Use **Visualize Layers** for paged layer-by-layer inspection and **3D View** for the complete build stack. If mismatch compensation was triggered, open **Mismatch Report** and inspect the affected layers.

Step 7 — Save and generate

Save Slice Data writes a JSON representation of the infill paths. **Generate LS Code** creates the Fanuc LS program. The GUI checks for an existing file before overwriting it.

Step 8 — Inspect the LS file

Use **Visualise LS** to parse the generated file and distinguish travel, normal weld and edge-compensation motion. Use **LS Offset Editor** only when you deliberately need a coordinate adjustment.

A good habit is: **analyse** → **slice** → **visualise** → **inspect reports** → **generate LS** → **visualise LS** → **only then consider robot-side execution**.

5. The slicing pipeline, from STL to LayerData

The core engine begins with `WAAMSLicer.load_stl()`. It validates the path, loads the mesh, handles a Trimesh Scene by concatenating its valid meshes, warns about non-watertight geometry, prints bounds and face/vertex counts, translates the base to $Z=0$, and automatically performs geometry analysis.

For every selected layer height, the engine intersects the mesh with a horizontal plane:

```
cross = mesh.section(  
    plane_origin=[0, 0, z],  
    plane_normal=[0, 0, 1]  
)
```

The resulting section is converted to planar geometry. One subtle but important implementation detail is that `to_planar()` can use an arbitrary local 2D origin. The slicer therefore recovers the X/Y translation from the returned transform and translates the Shapely polygons back into the world XY frame. Without this correction, successive layers can appear to shift even when the actual part is stationary.

Once valid polygons exist, each polygon is processed in the same order:

1. Compute wall paths.
2. Determine the region remaining after the walls.
3. If the region is empty, stop for this polygon.
4. Generate raster or contour infill.
5. Add any edge-compensation paths.
6. Store mismatch information.
7. Compute layer torch W/P/R.
8. Store optional per-layer weld speed.

This is the most important architectural choice in the current slicer: **walls are not an optional visual decoration around an independently generated fill**. The infill region is explicitly the material space left after wall generation.

6. Geometry analysis and parameter suggestions

The geometry analyser is intended as a practical pre-flight check rather than a full CAD feature-recognition system. It samples five sections at relative heights 0.10, 0.25, 0.50, 0.75 and 0.90 of the part height.

Each sampled polygon is inspected for holes and approximate minimum width. The analyser tracks solid polygons, annular polygons and layers containing multiple disconnected polygons. The current classifications are:

Type	Meaning in the current implementation
solid	Predominantly solid cross-sections with no dominant annular character.
thin_wall	Mostly solid sampled polygons whose minimum width is $\leq 1.5 \times$ the configured bead width.
annular	At least 70% of sampled polygon observations are annular.
multi_body	Multiple disconnected polygons occur in at least three sampled layers.
mixed	Neither solid, thin-wall, annular nor multi-body dominates; useful for transitions such as solid bases becoming hollow walls.

The minimum polygon width is estimated by binary searching the largest inward buffer radius for which the polygon remains non-empty, then doubling that radius. This is an approximate geometric width, not a local thickness map.

Suggestions are intentionally practical. For example, a thin wall may trigger a recommendation for one wall pass and no infill; an annular geometry may favour contour infill; a mixed geometry may be advised to use one wall and adaptive layers.

Important: these suggestions do not automatically change the user's configuration. They are advice displayed by the UI. The user remains responsible for deciding whether the recommendation makes sense for the actual WAAM process.

7. Layer generation: constant and adaptive modes

Constant mode is the simple case: Z levels are generated at a uniform spacing equal to `layer_height` from the mesh base toward its maximum Z.

Adaptive mode is more interesting. The current implementation uses a cusp-height relationship based on the surface normal:

```
t(z) = clamp( C_max / n_z , t_min , h_b )
```

Here `n_z` is an area-weighted 90th-percentile absolute Z component of the upward-facing facet normals intersecting the current Z plane. The intent is intuitive: a near-vertical surface has a small Z-normal component and is therefore not strongly constrained by cusp error; a near-horizontal upward surface has a large Z-normal component and receives a smaller layer thickness.

The implementation also explicitly requires `C_max < h_b` for adaptation to have an effect. If the cusp limit is too large, the calculated thickness is clipped to the maximum bead height and the schedule becomes effectively constant.

When the experimental bead model is enabled, the adaptive schedule is tied to physically observed bead-height bounds. The model is inverted to determine the travel speed required for each target layer thickness. The actual bead height at that assigned speed is then used to determine the next Z increment.

That distinction matters. The code does not blindly assume that the nominal maximum bead height is achieved at every adaptive layer. If a width constraint forces the torch to travel faster, the resulting bead can be shorter, and the code uses that actual modelled height to avoid an accumulating torch-to-deposit gap.

Adaptive mode can also account for a tilt-corrected vertical increment when adaptive torch tilt is active. In that case, the Z increment is reduced according to the cosine of the detected lean severity.

8. The experimental bead model

`waam_bead_model.py` is not a generic material database. It is a compact process model built from the project's DoE data. The code contains three speed-dependent lookup tables: multilayer steady-state bead height, single-layer bead width, and multilayer bead width.

Quantity	3.0 mm/s	5.0 mm/s	6.0 mm/s	8.0 mm/s
ML height (mm)	2.565	2.291	2.233	2.042
SL width (mm)	7.345	5.243	4.733	4.379
ML width (mm)	8.689	6.203	5.600	5.180

The full model uses speed samples from 3.0 to 8.0 mm/s in 0.5 mm/s increments and linearly interpolates between them. Inputs are clipped to the model's physical range. Height decreases with increasing travel speed, while the measured width tables also generally decrease with speed.

The model is deliberately **invertible**. It supports both directions:

Forward:

- speed → height
- speed → single-layer width
- speed → multilayer width

Inverse:

- target height → speed
- target single-layer width → speed
- target multilayer width → speed

The slicer uses these directions for different jobs. In adaptive layer mode, target bead height is converted to travel speed. The resulting speed is stored in `LayerData.weld_speed_mms` and later written to Fanuc `R[2]` at the beginning of that layer.

The slicer also uses the model's multilayer width at the assigned speed when choosing raster/contour spacing. This is a strong design choice because it means the path planner is not using a nominal six-millimetre bead while simultaneously commanding a process condition that produces a different bead width.

For edge compensation, the code can use the single-layer width model to find a faster/slower speed that produces a narrower pass suited to the remaining gap.

Validity rule: the model should be treated as valid only over the experimental process window from which it was obtained. The code clips inputs outside that window; clipping is not the same thing as experimental validation.

9. Wall generation

Wall planning is where the slicer becomes noticeably WAAM-specific. The core routine handles three useful situations: thin solid regions, thin annular regions, and ordinary thicker regions.

For a thin solid region whose minimum width is comparable to the bead width, the code attempts to generate a single centreline rather than repeatedly offsetting the boundary. For straight or gently curved strips it first uses the polygon's minimum rotated rectangle to find the long axis. If the geometry is too curved for that approximation, it falls back to an erosion/midpoint-style approach.

The thin-wall path is an open path. That is why the usual contour seam-rotation logic does not apply to it. Instead, the implementation can reverse the centreline direction on alternating layers.

For annular geometry, the wall can be represented by a closed ring around the outer boundary. Ordinary thicker regions can receive multiple inward wall passes.

The nominal wall step is based on bead width and overlap:

```
wall_step = bead_width × (1 - overlap_pct / 100)
```

With a 6 mm bead and 30% overlap, the nominal centre-to-centre spacing is 4.2 mm.

A key practical feature is that the code checks whether the requested number of wall passes can physically fit. If a section becomes too narrow, it can reduce the effective step or fall back to a narrower/thin-wall treatment rather than blindly placing the requested number of contours.

This is especially important for tapered geometry, where a wall that is perfectly reasonable at the bottom may become too narrow several layers later.

10. Infill strategies

After walls are generated, the slicer computes the remaining interior region. The user can choose raster or contour infill.

Raster infill

Raster creates parallel deposition lines through the remaining polygonal region. The base angle is `raster_angle`, and the layer-to-layer angle is incremented by `angle_increment`. A common setting is 0°/90° alternation.

The raster planner supports several line-order strategies: sequential, alternating, interleaved, random and custom at the core-API level. The GUI exposes sequential, alternating, interleaved and random. Direction can independently be same, boustrophedon or random.

Boustrophedon means adjacent lines are traversed in opposite directions. In practice this reduces unnecessary retract/reposition behaviour between neighbouring raster lines.

The minimum path-length filter is a WAAM-oriented quality control. Very short raster stubs near boundaries can be undesirable because they create tiny deposition events. Paths shorter than `min_path_length_frac × bead_width` are therefore discarded.

Contour infill

Contour infill repeatedly offsets the remaining region inward and converts those regions into ring-like toolpaths. The current configuration forces inside-to-outside deposition order in the GUI. This is intended to avoid the height-accumulation tendency associated with depositing from the outside toward the centre.

Each contour ring receives a seam angle based on the layer seam angle plus a small per-ring offset. This prevents all rings in the same layer from starting at exactly the same angular location.

The two strategies serve different geometries. Raster is natural for bulk solid fills; contour is often more natural for annular or curved-wall regions.

11. Gap / fill-mismatch compensation

This is another part of the slicer that exists specifically because WAAM beads are large relative to many cross-sections. Even if the nominal spacing is correct, the number of complete beads that fit inside a finite region may leave a small residual gap.

The slicer records a `MismatchInfo` object containing the approximate region width, number of paths, filled width, remaining gap, strategy used and any edge-pass speed.

Two main approaches are used.

Strategy 1 — adjusted spacing. When the residual gap is small enough, the planner can redistribute the paths over the available width so the existing passes cover the region more evenly. The summary reports the effective overlap produced by this adjustment.

Strategy 2 — extra edge/centre pass. When spacing adjustment is not enough or the remaining region is larger, the planner retains the normal paths and adds a compensation path through the remaining region. When the DoE bead model is available, its single-layer width inversion is used to choose a speed for that pass.

The controlling threshold is expressed as a fraction of bead width. With the default 0.5, a gap below half a bead width is considered small enough to attempt spacing adjustment.

The resulting compensation passes are kept separate from normal infill in `LayerData.edge_passes`. The LS generator can therefore write them at their explicit speed rather than treating them as ordinary weld moves.

This separation also makes the mismatch report useful. The UI shows affected layers, gap size, strategy and edge-pass speed and can export the table as CSV.

12. Seam management and path ordering

For closed contour paths, seam location is controlled independently of infill direction. The available modes are fixed, rotate and random.

Mode	Behaviour
fixed	Use one fixed angular location for every layer.
rotate	Advance the seam by <code>seam_rotate_deg</code> each layer; current default is 67°.
random	Use a deterministic pseudo-random angle derived from the layer index.

The intent of rotating the seam is simple: avoid building a visible vertical start/stop defect at one location. The code also offsets the seam between contour rings inside the same layer.

For thin-wall centreline paths, seam rotation is not meaningful because the path is open. The code instead alternates direction between layers when `thin_wall_alternating` is enabled.

Line order and direction are separate concepts. *Line order* decides which raster line is deposited next. *Direction mode* decides which way the torch travels along that line. Keeping these separate makes it possible to change heat distribution without rewriting the geometry generation logic.

The current core API also contains a CUSTOM line-order enum, although the GUI does not expose a custom-order editor. This is a good example of why the README distinguishes GUI parameters from programmatic capabilities.

13. Torch orientation and adaptive tilt

Torch orientation is stored per layer as (W, P, R). The default behaviour is conservative: when torch tilt is disabled, the layer retains the configured base orientation.

Fixed tilt mode

When torch tilt is enabled but adaptive tilt is off, the same W, P and R values are used for every layer. This is appropriate when the whole part is deliberately mounted at an angle or when a fixed process orientation is known experimentally.

Adaptive directional tilt

Adaptive tilt is intentionally not based on simply choosing the strongest mesh normal. The code comments explain why: for symmetric parts such as annular or dome-like geometry, a single normal can point in an arbitrary direction from one layer to the next. That could create unnecessary wrist rotation without helping the overall build.

Instead, the implementation looks for a *dominant direction* of overhang. It uses two complementary methods:

- **Method A — mesh face normals:** finds downward-facing surface normals, rejects several characteristic marching-cubes artefact normal magnitudes, computes an area-weighted XY direction and checks directional consistency.
- **Method B — centroid shift:** compares the XY centroid of two horizontal sections separated by 2.5 mm. This is intended to detect a consistently leaning wall even when its surface normals do not contain downward-facing faces.

The tilt only activates when the direction is sufficiently consistent. The default consistency threshold is 0.65, meaning at least 65% of the relevant overhang area should support one direction.

The detected lean is decomposed into W and P components, capped by the configured maximum tilt, and W changes are rate-limited between layers. R remains fixed.

This is best viewed as a geometry-aware assist, not a substitute for robot reachability and collision analysis. A valid geometric tilt can still be invalid for a particular robot configuration.

14. Fanuc LS generation

The LS generator takes the completed LayerData list and writes a Fanuc R-30iB LS program. Its job is intentionally mechanical: translate planned paths and layer metadata into robot-readable motion and weld commands.

The program begins with a Fanuc header, register initialisation and the configured tool/frame information. The default process registers are:

Register	Meaning in the project
R[1]	Non-weld / air travel speed
R[2]	Normal weld travel speed
R[3]	Interlayer dwell time

Within a deposition segment the sequence is essentially:

```
approach start point at travel speed
Weld Start[S1,S2]
weld points at R[2] (or explicit edge-pass speed)
Weld End[S1,S2]
```

Between segments in the same layer, the generator can move at R[1] to the next start point. It does not insert a Z lift between normal segments; the project assumes wire retraction/process behaviour is sufficient to isolate segments.

Between layers, behaviour depends on `lift_clearance_mm`. With a positive clearance, the generator lifts, travels horizontally, and descends. With zero clearance, it moves directly to the next layer's start point. The source explicitly notes that zero lift is only appropriate when wire retract is reliable.

Adaptive weld speed is written before the layer dwell so that R[2] is already correct when deposition begins. Fanuc move statements require integer mm/s for explicit edge-pass speeds, so those speeds are rounded before being written.

Coordinates are written in the STL frame. The generator intentionally does not apply a robot-world transform; the separate LS Offset Editor exists for real-world coordinate offsets.

15. LS inspection and coordinate offsets

The LS tools close the loop between generation and robot code. The parser reads position blocks, motion statements, weld-start/end markers and layer comments into structured objects.

Object	Purpose
LSPoint	One P[n] position with X/Y/Z/W/P/R.
LSMove	One motion reference with line number, point number, speed string, weld state and layer.
LSLayer	Collection of motions belonging to one layer.
ParsedLS	Complete parsed program plus errors/warnings.

Motion classification is simple and transparent: R[1] is treated as travel, R[2] as normal weld, and explicit mm/sec speeds as edge-compensation motion.

The LS Visualiser presents the toolpath in 3D, provides point/move/layer statistics and allows travel visibility and layer selection to be controlled. It can also show torch arrows and save the current view or per-layer views.

The LS Offset Editor works directly on the /POS section of an LS file. It can apply deltas to X, Y, Z, W, P and R, preview the coordinate range, and save the modified program. The intent is to keep robot setup offsets separate from the geometry coordinates generated by the slicer.

Use this distinction carefully: an offset is a coordinate transformation, not a collision check, kinematic check, or robot reachability check.

16. Outputs and reports

The current UI produces several useful outputs.

Output / tool	What it contains
slice_data.json	Layer index, Z height and infill path coordinates exported by the GUI.
Fanuc .LS	Robot program containing motion points, weld commands, registers and layer sequencing.
Mismatch Report CSV	Layer, Z, region width, number of paths, filled width, gap, strategy and edge speed.
Slicer console summary	Layer schedule, mismatch statistics, overhang criticality and related process information.
Layer Viewer	Paged 2D inspection of wall, infill, edge-pass and torch-orientation information.
3D Build View	Stacked 3D representation of generated wall/infill/edge paths.
LS Visualiser	Parsed LS toolpath with travel/weld/edge classification and layer statistics.

The core `print_summary()` function can also export a CSV report programmatically through its `export_csv` argument. The report tracks layer thickness, weld speed, path counts, compensation strategies, maximum gap and overhang angle.

The overhang section divides layers into safe, caution and critical categories relative to the configured overhang limit. The source also suggests increasing dwell at caution/critical layers; treat that as a process recommendation rather than an automatic process-control guarantee.

17. Configuration reference

The following table focuses on the parameters that materially affect slicing. The GUI has additional Fanuc and process-calculator settings, which are included below.

Layer and bead geometry

Parameter	Default in core	Purpose
layer_height	2.0 mm	Constant layer thickness.
adaptive_layers	False	Enable geometry-aware layer scheduling.
bead_height	2.565 mm	Maximum bead-height bound used without the experimental model.
overhang_limit_deg	20°	Overhang reporting/flagging threshold.
cuspl_height_max	1.0 mm	Cusp-height constraint for adaptive layer scheduling.
use_bead_model	True	Tie adaptive slicing to the DoE bead model.
bead_width	6.0 mm	Nominal bead width and geometry reference.
overlap_pct	30%	Bead-to-bead overlap used to derive spacing.

Walls, infill and compensation

Parameter	Default	Purpose
num_contours	2	Number of requested wall passes.
seam_mode	rotate	Fixed, rotating or random contour seam.
seam_rotate_deg	67°	Layer-to-layer seam rotation.
thin_wall_alternating	True	Reverse open thin-wall paths on alternate layers.
infill_type	raster	Raster or contour.
raster_angle	0°	Base raster orientation.
angle_increment	90°	Layer-to-layer raster rotation.
direction_mode	boustrophedon	Line travel direction.
line_order	interleaved	Order of raster lines.
min_path_length_frac	0.5	Reject raster stubs shorter than this bead-width fraction.
mismatch_threshold_frac	0.5	Threshold for fill-gap compensation strategy.
edge_speed_multiplier	1.5	Fallback edge-pass speed factor when no bead model is available.

18. Python API and programmatic use

The GUI is convenient, but the core is usable directly from Python. A minimal constant-layer workflow is:

```
from waam_slicer import WAAMSlicer, SlicerConfig

cfg = SlicerConfig(
    layer_height=2.5,
    bead_width=6.0,
    overlap_pct=30.0,
    num_contours=1,
    infill_type=InfillType.RASTER,
)

slicer = WAAMSlicer(cfg)
slicer.load_stl("part.stl")
slicer.slice()
slicer.print_summary()
```

For programmatic use, import the enums explicitly:

```
from waam_slicer import (
    WAAMSlicer, SlicerConfig,
    InfillType, LineOrder,
    DirectionMode, SeamMode
)
```

Adaptive mode can be configured through the same object. When the bead model is available and enabled, the resulting layer objects carry per-layer weld speeds.

Generating LS code directly:

```
from waam_ls_generator import LSConfig, LSGenerator

ls_cfg = LSConfig(
    prog_name="WAAM_PROG",
    uframe=0,
    utool=6,
    travel_speed_val=50.0,
    weld_speed_val=4.0,
    dwell_val=10.0,
    reduction_mode="reduced",
    curve_tolerance_mm=0.5,
)

gen = LSGenerator(ls_cfg)
gen.save("WAAM_PROG.LS", slicer.layers)
```

The important conceptual point is that `LSGenerator` consumes the slicer's output. If you build your own path planner later, you can still target the same `LayerData`-style interface and reuse the LS generator.

19. Practical configuration recipes

These are starting configurations, not universal process recipes. The correct settings still depend on the actual WPS, wire, material, robot, torch and experimentally validated bead behaviour.

A. Conventional solid wall / bulk part

Adaptive layers:	off
Bead width:	6.0 mm
Overlap:	30%
Wall passes:	1-2
Infill:	raster
Raster angle:	0°
Angle increment:	90°
Direction:	boustrophedon
Line order:	interleaved
Seam:	rotate

This is the straightforward choice when the cross-section has enough room for several bead tracks.

B. Hollow / annular part

Wall passes:	0 or 1
Infill:	contour
Seam:	rotate
Contour order:	inside → outside

The point is to treat the wall as a wall rather than accidentally filling the hole.

C. Thin single-bead wall

Wall strategy:	single centreline
Bead width:	match actual wall thickness
Thin-wall alternating:	on
Infill:	none / walls only

The core slicer automatically contains thin-wall handling, but the GUI does not offer a literal “none” infill type. In practice, the geometry suggestions should guide the user toward a configuration where the wall paths cover the section and no meaningful interior remains.

D. Curved geometry where surface error matters

Adaptive layers:	on
Use bead model:	on
Cusp height C _{max} :	tune experimentally
Overhang limit:	process-dependent

Here the main benefit is that the layer schedule is allowed to change with surface orientation, while the experimental bead model ties the schedule to achievable deposition height and speed.

E. Deliberately tilted part

Torch tilt:	on
Adaptive tilt:	off
W/P base:	set from the actual setup
R base:	fixed torch spin

Use adaptive tilt only when the geometry really calls for directional, layer-varying orientation.

20. Validation, limitations and safe deployment

The slicer is research-oriented software. A successful slice or a syntactically valid LS file does not prove that the resulting program is safe or that it will produce the intended deposit.

Mesh and geometry limitations

- STL quality matters. Non-watertight meshes are allowed but can produce gaps or missing sections.
- The primary slicer is planar: it intersects horizontal Z planes. It is not a general non-planar slicer.
- Geometry analysis is sampled and approximate; it is not a full feature-recognition or manufacturability engine.
- Thin-wall centreline extraction is intentionally heuristic for complex curved shapes.

Process-model limitations

- The bead model is derived from experimental data and should not be extrapolated casually outside 3–8 mm/s.
- Linear interpolation between measured points is a compact approximation, not a mechanistic weld model.
- The model does not automatically account for every factor that changes bead geometry — for example thermal history, local curvature, shielding, current/voltage changes or transient start/stop behaviour.

Robot and LS limitations

- The generated coordinates are in the STL frame. The generator does not perform a complete robot-world calibration transform.
- The LS Offset Editor applies coordinate deltas but does not perform collision, reachability or singularity analysis.
- Torch orientation values must be consistent with the actual Fanuc configuration and tool definition.
- Zero interlayer lift is explicitly dependent on reliable wire/process retraction.

Recommended validation ladder

1. Validate STL orientation and Z=0 placement.
2. Run geometry analysis and review suggestions.
3. Slice and inspect 2D layers.
4. Inspect the 3D stack.
5. Review mismatch and overhang reports.
6. Generate LS.
7. Parse and visualise the LS file.
8. Check robot frame/tool configuration independently.
9. Dry-run / simulate the robot program using the actual robot setup.
10. Validate a conservative physical deposition before scaling up.

The project contains enough process-aware logic to make this workflow practical, but it should remain inside a controlled engineering validation process before being treated as production robot software.

21. Developer notes and extension points

The current separation gives several clean places to extend the project.

Bead model extension. Replace or augment the lookup tables with a richer response surface while keeping the same forward/inverse interface. The slicer only needs functions that map speed to geometry and geometry to speed.

Process-aware spacing. The current code already uses modelled multilayer width when adaptive bead-model slicing is active. A future version could make overlap depend explicitly on bead geometry rather than a single fixed percentage.

Adaptive layer constraints. The cusp-height logic could be combined with explicit minimum/maximum layer-height constraints, local curvature estimates or a validated deposition stability map.

Robot-aware planning. The cleanest future boundary is after LayerData: add a robot kinematics/reachability stage before LS generation. That stage could reject or modify orientations that are geometrically valid but robotically impossible.

Path optimisation. The project already separates line order, direction and seam placement. That makes it a good place to experiment with heat-aware ordering, travel minimisation or region-by-region scheduling without rewriting the geometry engine.

Output verification. The LS parser can be extended into a stronger static checker: duplicate points, excessive speed changes, unexpected Z motion, invalid frame/tool references, register usage and controller memory estimates are natural additions.

One useful design principle to preserve is the current boundary between **geometry**, **process** and **robot syntax**. Keeping those layers separate will make the project much easier to maintain as the process model becomes more sophisticated.

22. Troubleshooting

Symptom	Likely cause / first check
STL not found	Check the selected path; the core loader raises <code>FileNotFoundError</code> with the path.
Few or zero layers	Check mesh Z extent, layer height, watertightness and whether the horizontal sections are valid.
Layers appear shifted	The current core already corrects the planar transform; if this persists, inspect the input mesh/orientation and visualisation.
Unexpected gaps in fill	Open Mismatch Report. Check bead width, overlap, threshold and whether the experimental bead model is active.
Thin wall is overfilled	Run Geometry Analysis. Check whether the wall thickness is close to bead width and reduce wall passes / meaningful infill accordingly.
Adaptive layers do not change	Check that adaptive mode is on and <code>C_max</code> is below the bead-height upper bound. Also check the bead-model availability.
Adaptive speed seems odd	Remember that the experimental model is clipped to its validated 3–8 mm/s range.
LS file is very large	Use reduced/custom point reduction and increase curve tolerance carefully. The generator reports point count and file size.
LS visualiser shows no weld moves	Inspect whether the LS file contains the expected <code>Weld Start/End</code> and <code>R[2]</code> motion statements.
Robot position needs shifting	Use the LS Offset Editor only after independently deciding the required <code>X/Y/Z/W/P/R</code> deltas.
Torch motion changes unexpectedly	Check whether adaptive tilt is enabled and whether the detected geometry has a dominant directional overhang.

If a problem is difficult to diagnose, the console output is worth reading first. The slicer intentionally reports bounds, layer counts, wall/path counts, adaptive speeds, mismatch statistics, overhang criticality and LS generation statistics.

For developers, a second useful debugging technique is to stop before LS generation and inspect the `LayerData` objects directly. That keeps geometry/path issues separate from Fanuc formatting issues.

Closing note

The strongest part of this slicer is not any single algorithm. It is the way the pieces are connected: the geometry determines the available deposition space; the experimentally derived bead model gives that geometry a process-aware scale; the path planner accounts for finite bead width; and the LS generator carries the resulting decisions all the way to robot code. The visualisation and parsing tools then give you a chance to inspect the result before it reaches the machine.

That makes the project a useful research platform as well as a slicer: individual assumptions can be replaced, tested and compared without having to rebuild the entire pipeline.

Source basis: the five current project files supplied with this project: `waam_slicer.py`, `waam_bead_models.py`, `waam_slicer_ui.py`, `waam_ls_generator.py` and `waam_ls_tools.py`.